

JAVA INTERVIEW PREPARATION

CHAPTER 74

GREEDY ALGORITHMS,
INTERVALS, AND SCHEDULING

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Greedy Algorithms, Intervals, and Scheduling

Senior / Lead Java Developer Reading Chapter

A greedy algorithm makes the best-looking local choice and commits without reconsidering earlier decisions. Sometimes that produces a globally optimal result; sometimes it produces an attractive but wrong shortcut. The difference is a proof, not intuition.

Senior interviews use greedy problems to test whether a candidate can identify an invariant, construct counterexamples, choose a sorting key, define interval boundaries, and distinguish offline optimality from production scheduling concerns such as fairness, deadlines, preemption, and changing input.

A local rule needs a global argument

Greedy algorithms usually rely on one of these proof styles:

- exchange argument: transform an optimal solution so it begins with the greedy choice without making it worse;
- stays-ahead argument: after every step, the greedy partial solution is at least as good as any competitor under a defined measure;
- cut property: the cheapest safe edge crossing a partition belongs to some optimum;
- matroid-style structure: independent sets support safe exchanges;
- contradiction: assume the first greedy divergence and replace it.

```
candidate choices
  |
  v
locally best under rule
  |
  +-- proof says an optimum can contain this choice
  v
reduced remaining problem
```

If the local choice changes the future in a way the proof cannot control, dynamic programming or search may be required.

Interval semantics come first

Intervals appear in meeting scheduling, resource reservations, deployments, maintenance windows, and event streams. Define whether endpoints are closed, open, or half-open.

```
[start, end)

meeting A: [09:00, 10:00)
meeting B: [10:00, 11:00)

A and B do not overlap under half-open semantics.
```

Half-open intervals compose naturally and allow zero-length intervals, but a business domain may define touching reservations differently because cleanup or travel time is required. Convert time zones and apply buffers before the algorithm; do not hide business semantics inside comparison operators.

Use immutable interval values and safe comparators:

```
record Interval(long start, long end) {
    Interval {
        if (end < start) throw new IllegalArgumentException("reversed");
    }
}

Comparator<Interval> byStartThenEnd = Comparator
    .comparingLong(Interval::start)
    .thenComparingLong(Interval::end);
```

Maximum number of nonoverlapping intervals

To select the largest number of nonoverlapping intervals on one resource, sort by earliest finish time and repeatedly accept the next interval whose start is not before the last selected end.

```
time --->

A: [-----]
B:  [--]
C:   [---]
D:    [--]

earliest-finish selection can choose B, C, D
choosing A first blocks more future intervals
```

```
static List<Interval> maximumCompatible(List<Interval> input) {
    List<Interval> sorted = new ArrayList<>(input);
    sorted.sort(Comparator.comparingLong(Interval::end)
        .thenComparingLong(Interval::start));

    List<Interval> selected = new ArrayList<>();
    long availableAt = Long.MIN_VALUE;
    for (Interval interval : sorted) {
        if (interval.start() >= availableAt) {
            selected.add(interval);
            availableAt = interval.end();
        }
    }
    return selected;
}
```

The exchange proof is direct. Consider an optimal schedule's first interval. The greedy interval finishes no later. Replacing the optimum's first interval with the greedy one cannot make any later interval infeasible. Therefore some optimum begins with the greedy choice, and the argument repeats on the remainder.

Sorting by shortest duration or earliest start is not generally correct. A short interval placed late can block two earlier opportunities; an early-starting long interval can block many short ones.

Tie-breakers do not change the maximum count when finish times are equal under the proof, but they affect which optimal schedule is returned. Choosing the later start may leave a shorter occupied span; choosing a stable ID makes output reproducible. If downstream behavior depends on selected identities, tie-breaking is part of the contract rather than cosmetic sorting.

Weighted interval scheduling is not greedy

If intervals have values and the goal is maximum total value, earliest finish no longer suffices.

```
three compatible low-value jobs: 4 + 4 + 4 = 12
one overlapping high-value job:          20
```

The weighted problem is commonly solved with dynamic programming. Sort by finish time, compute $p(i)$, the last interval compatible with i , then use:

```
dp[i] = max(dp[i-1], value[i] + dp[p(i)])
```

This counterexample is as important as the greedy solution. Similar-looking requirements can cross the boundary from safe local choice to DP.

Merging intervals

Merging overlapping intervals is greedy after sorting by start. Maintain the last merged interval. If the next interval overlaps, extend the end; otherwise begin a new merged region.

```
static List<Interval> merge(List<Interval> input) {
    if (input.isEmpty()) return List.of();
    List<Interval> sorted = new ArrayList<>(input);
    sorted.sort(Comparator.comparingLong(Interval::start)
        .thenComparingLong(Interval::end));

    List<Interval> output = new ArrayList<>();
    long start = sorted.get(0).start();
    long end = sorted.get(0).end();

    for (int i = 1; i < sorted.size(); i++) {
        Interval next = sorted.get(i);
        if (next.start() <= end) { // change for non-touching semantics
            end = Math.max(end, next.end());
        } else {
            output.add(new Interval(start, end));
            start = next.start();
            end = next.end();
        }
    }
    output.add(new Interval(start, end));
    return output;
}
```

The comment exposes the domain decision. Under strict half-open overlap, touching intervals satisfy `next.start() == end` and do not overlap, so the condition would use `<`. Some aggregation tasks intentionally coalesce touching ranges.

Time is $O(n \log n)$ for sorting and $O(n)$ for scanning. If input is already sorted and that precondition is trustworthy, scanning is linear.

Minimum meeting rooms

The maximum number of simultaneously active intervals is the minimum number of identical rooms required. One method sorts starts and ends separately.

```
events at one timestamp under [start, end):

process END before START
so a room released at 10:00 can host a meeting starting at 10:00
```

```

static int rooms(List<Interval> meetings) {
    long[] starts = meetings.stream().mapToLong(Interval::start).sorted().toArray();
    long[] ends = meetings.stream().mapToLong(Interval::end).sorted().toArray();
    int s = 0, e = 0, active = 0, maximum = 0;

    while (s < starts.length) {
        if (starts[s] < ends[e]) {
            active++;
            maximum = Math.max(maximum, active);
            s++;
        } else {
            active--;
            e++;
        }
    }
    return maximum;
}

```

Empty or zero-length meeting policy needs care; production code should validate the exact contract before indexing `ends`. A min-heap of current room end times can also assign concrete room IDs: reuse the room with the earliest end when compatible, otherwise allocate another.

The event-sweep technique generalizes to maximum concurrent requests, capacity usage, and overlapping subscriptions. If events have quantities rather than unit counts, sort signed deltas with deterministic same-time ordering.

If rooms have different capabilities or meetings require specific equipment, maximum overlap is only a lower bound. Assignment becomes a matching, flow, constraint-search, or optimization problem depending on the model. Likewise, allowing meeting movement, splitting, or preemption changes the problem. Do not stretch the identical-resource greedy proof beyond its assumptions.

Minimum points to cover intervals

To hit every closed interval with the fewest points, sort by end and choose the end of the first interval not already covered. That point extends as far right as possible while still covering the current interval.

The same idea solves minimum arrows for closed balloon intervals. Boundary semantics matter: an arrow at endpoint `x` covers intervals containing `x`, but half-open intervals ending at `x` would not contain it.

```

intervals: [1,4], [2,5], [6,8]
choose 4 -> covers first two
choose 8 -> covers third

```

The exchange argument replaces an optimal first covering point with the earliest finishing endpoint without losing coverage of any interval that could be grouped with the first.

Fractional versus 0/1 knapsack

Fractional knapsack permits taking part of an item. Sort by value-to-weight ratio and take the best ratios first. The exchange argument says any lower-ratio weight in a solution can be exchanged for available higher-ratio weight to improve value.

```

fractional: take 100% of item A, then 40% of item B
0/1:       item B must be taken entirely or not at all

```

The same rule fails for 0/1 knapsack because indivisibility prevents a small exchange. Ratio comparisons should avoid floating-point error where exact decisions matter: compare `valueA * weightB` and `valueB * weightA` using overflow-safe wider arithmetic or `BigInteger` for extreme values.

This pair is a classic reminder that one changed constraint can invalidate the greedy proof.

Huffman coding

Huffman coding repeatedly merges the two least frequent symbols or subtrees. A min-priority queue exposes them.

```
frequencies: A:5, B:7, C:10, D:15
```

```
merge 5 + 7 -> 12
merge 10 + 12 -> 22
merge 15 + 22 -> 37
```

The resulting prefix code minimizes weighted code length under the model. More frequent symbols tend to receive shorter codes. Equal frequencies can produce different but equally optimal trees; deterministic file formats need a canonical Huffman representation or stable tie-breakers.

Real compression formats also need headers, bit packing, corruption handling, and sometimes block adaptation. The tree algorithm alone is not a complete codec.

Jump Game and maintaining a frontier

For the reachability version of Jump Game, scan left to right while maintaining the furthest reachable index.

```
static boolean canReachEnd(int[] jump) {
    int furthest = 0;
    for (int i = 0; i < jump.length; i++) {
        if (i > furthest) return false;
        long candidate = (long) i + jump[i];
        furthest = (int) Math.min(jump.length - 1L,
            Math.max(furthest, candidate));
        if (furthest >= jump.length - 1) return true;
    }
    return jump.length == 0 || furthest >= jump.length - 1;
}
```

The invariant is that every index up to `furthest` is reachable through some choices seen so far. It does not commit to one path; it compresses the reachable frontier. Negative jumps would violate the problem model and should be rejected if input is not trusted.

The minimum-jumps variant can process reachable layers greedily, similar to BFS over implicit edges: when the current layer ends, commit one jump and extend to the furthest next boundary.

Gas station and eliminating starts

In the circular gas-station problem, if total gas is less than total cost, no solution exists. Otherwise, scan while maintaining a tank balance. When balance becomes negative at position `i`, no start between the current candidate and `i` can work, so the next candidate is `i + 1`.

```
static int startStation(int[] gas, int[] cost) {
    if (gas.length != cost.length) throw new IllegalArgumentException();
    long total = 0, tank = 0;
    int candidate = 0;
    for (int i = 0; i < gas.length; i++) {
        long delta = (long) gas[i] - cost[i];
        total += delta;
        tank += delta;
        if (tank < 0) {
            candidate = i + 1;
            tank = 0;
        }
    }
    return total >= 0 && gas.length > 0 ? candidate : -1;
}
```

The proof is the elimination argument. Any later start inside the failed segment begins without the earlier surplus and therefore also fails by i .

Greedy change and counterexamples

Choosing the largest coin first works for some currency systems but not arbitrary denominations.

```
coins: 1, 3, 4    target 6

greedy: 4 + 1 + 1 = 3 coins
optimal: 3 + 3    = 2 coins
```

Canonical coin systems have special structure; a general minimum-coin problem needs dynamic programming or shortest-path reasoning. When an interviewer proposes a greedy rule, actively search for a small counterexample with competing local and global effects.

Scheduling with deadlines and profits

Scheduling changes substantially with the model:

- maximum number of compatible intervals: earliest finish greedy;
- weighted compatible intervals: dynamic programming;
- unit-time jobs with deadlines and profits: sort by profit and place each job in the latest available slot before its deadline;
- minimize maximum lateness on one machine without release times: earliest deadline first;
- arbitrary processing times, release times, preemption, and multiple machines: different and often harder problems.

Do not say "sort by deadline" until the objective and constraints are explicit. Maximum throughput, total profit, average completion time, and worst lateness can require different rules.

Union-Find can accelerate finding the latest free slot in unit-time job sequencing. A simpler boolean-slot solution may be preferable when deadlines are small and clarity matters.

Greedy graph algorithms

Dijkstra repeatedly finalizes the smallest tentative distance under nonnegative weights. Prim repeatedly chooses the cheapest edge connecting the current tree to a new vertex. Kruskal chooses the globally cheapest edge that does not create a cycle.

These are greedy algorithms because each commits a locally safe choice backed by a proof: finalized distance, cut property, or component exchange. The weight assumptions and safe-edge definitions are

essential. Negative edges break Dijkstra's finalization argument.

Online versus offline greedy decisions

Most interview interval algorithms are offline: all intervals are available for sorting. Production schedulers often receive work online. Without future knowledge, an algorithm may not achieve the offline optimum. Competitive analysis compares an online policy with an ideal offline solution.

Admission control may reject a low-value long job now to preserve uncertain future capacity, or accept it to avoid idle resources. The system also needs fairness, tenant quotas, cancellation, and service-level objectives. A mathematically optimal offline schedule may be impossible to implement when jobs arrive, change, or fail.

Greedy policies can create starvation. Strict priority scheduling may never serve low-priority work. Aging, weighted fair sharing, reservation, or maximum wait limits deliberately modify the local choice to satisfy system objectives.

Replanning also has a cost. Recomputing an optimal schedule after every new job can move already communicated assignments and erode user trust. A production objective may include both schedule quality and churn penalty. Freezing a near-term horizon, applying changes only beyond a boundary, or requiring a minimum improvement before replacement creates stability at the cost of theoretical optimality.

Production interval and scheduling concerns

Time intervals require timezone and clock semantics. Store timeline instants for cross-zone coordination, keep original zone when wall-clock intent matters, and decide how daylight-saving gaps or overlaps are resolved. Use half-open intervals internally where possible.

Scheduling state changes concurrently. Two nodes can both see a slot as free and allocate it unless reservation is atomic. A greedy selection algorithm does not solve distributed coordination. Use a transactional constraint, conditional update, lease, or single owner, and define behavior under retries and expiration.

Idempotency is required when a scheduler retries after an uncertain response. A reservation command should carry a stable request ID, and the database must prevent the same logical job from consuming two slots. Leases need fencing or version checks so a delayed former owner cannot overwrite a newer assignment after its lease expires.

Measure schedule utilization, wait-time percentiles, deadline misses, preemptions, starvation age, rejection, and fairness by tenant or class. Recompute policies can thrash if every small input change replaces the schedule; use stability costs or bounded replanning.

Testing greedy algorithms

For every greedy implementation, build a brute-force or DP oracle for small input and compare random cases. Test empty input, one item, equal endpoints, touching intervals, zero-length ranges, duplicate intervals, negative or extreme weights where allowed, overflow, and multiple optimal solutions.

Verify properties:

- selected intervals are compatible under the declared boundary rule;
- no brute-force schedule has more intervals on small inputs;
- merged intervals are ordered and nonoverlapping;
- room count equals maximum overlap from an event oracle;

- a covering point lies in every assigned interval;
- greedy and DP results intentionally diverge on known counterexamples;
- sorting does not mutate caller data unless documented.

Counterexample-driven testing is especially important. Preserve the smallest case that disproves an incorrect local rule.

Common senior-level traps

- Calling a local rule optimal without a proof.
- Using earliest start instead of earliest finish for interval selection.
- Applying unweighted interval greedy to weighted intervals.
- Ignoring whether touching intervals overlap.
- Processing starts before ends at equal half-open timestamps.
- Using floating-point ratios when exact ordering matters.
- Applying fractional-knapsack greedy to 0/1 items.
- Assuming largest-coin-first works for arbitrary denominations.
- Naming a scheduling rule before defining the objective and constraints.
- Treating an offline optimum as an online production policy.
- Ignoring starvation, atomic reservation, time semantics, and replanning stability.

A strong interview answer

I first define the objective, constraints, and interval or equality semantics. Then I state the greedy choice and prove it with an exchange, stays-ahead, cut, or elimination argument. I actively test the rule against small counterexamples and identify which changed constraints invalidate the proof, such as weights in interval scheduling or indivisible items in knapsack. I explain sorting and scanning complexity, stable tie-breakers, and overflow. In production I separate offline selection from online admission and add fairness, atomic reservation, time-zone, cancellation, and observability requirements.

Chapter summary

Greedy algorithms are safe local commitments backed by global structure. Earliest finish maximizes compatible interval count; start-order scanning merges ranges; event sweeps measure concurrency; rightmost endpoints cover intervals; ratios solve fractional but not 0/1 knapsack; and graph greedy choices rely on weight and cut invariants. Counterexamples define the boundary of each proof. Production scheduling adds online uncertainty, fairness, atomicity, and temporal semantics beyond the textbook optimum.

Revision Notes

- Greedy correctness requires a proof, not a plausible local rule.
- Common proofs use exchange, stays-ahead, cut, or elimination arguments.
- Define open/closed/half-open boundaries before interval code.
- Earliest finish maximizes the number of compatible unweighted intervals.
- Weighted interval scheduling requires DP with previous-compatible states.
- Merge intervals by start order and extend the active end.
- Decide whether touching ranges merge.
- Maximum overlap equals minimum rooms for identical resources.
- Under half-open semantics, process an end before a start at the same time.
- Choosing earliest endpoints minimizes points covering closed intervals.
- Value/weight greedy is correct for fractional, not 0/1, knapsack.
- Huffman repeatedly merges the two least frequent nodes.
- Jump Game maintains a furthest-reachable frontier.
- Gas-station failure eliminates every start in the failed segment.
- Largest-coin-first fails for arbitrary denominations.
- Scheduling rules depend on objective, duration, release, deadline, machine, and preemption constraints.
- Offline optimality does not imply an optimal online policy.
- Production scheduling needs fairness, atomic reservation, time semantics, and limits.

Likely interview question: How do you know a greedy algorithm is correct?

State the local choice and prove that some optimal solution can include it, or that the greedy partial solution stays at least as good after every step. If you cannot prove the exchange or invariant, search for a counterexample or use DP/search.

Likely interview question: Why earliest finish for interval scheduling?

It leaves the largest remaining time region. Replacing the first interval in an optimal schedule with the earliest-finishing compatible interval cannot invalidate later selected intervals, so an optimum exists with the greedy first choice.

Likely interview question: Why does weighted interval scheduling need DP?

Maximizing count and maximizing value are different. One long interval may outweigh many short intervals, so earliest finish is not enough. DP compares excluding interval i with including its value plus the best result before its compatible predecessor.

Likely interview question: What changes in a production scheduler?

Jobs arrive and fail online, reservations require atomicity, clocks and zones affect deadlines, priorities can starve work, and objectives include fairness and SLA compliance. Treat the greedy algorithm as one policy component, not the entire system.